Learning Equilibrium Polymer Distributions with Continuous Normalizing Flows and Diffusion Models

Author Name
Affiliation
(Dated: May 11, 2026)

Our long-term goal is to learn models that infer molecular trajectories from static structural samples, including protein conformational ensembles. We first study controlled Brownian-dynamics synthetic systems where we can evaluate the equilibrium distribution and score field directly. The codebase uses Rouse-chain simulations as the main data source and compares two likelihood- or score-based models: an equivariant continuous normalizing flow (CNF) and an equivariant variance-preserving diffusion model. We train both models on centered coordinate snapshots and evaluate generated-configuration quality and agreement between learned log-density gradients and Brownian drift forces.

I. MOTIVATION

Our long-term goal is to develop generative models that infer molecular conformational dynamics from static structural samples. This problem matters especially for proteins, where experimental methods can provide structural ensembles but often omit the time ordering or transition information needed to reconstruct dynamics directly. Atomistic proteins make this idea difficult to evaluate directly because experiments do not fully observe the effective dynamics or provide the correct score or force field as a clean target.

We therefore begin with controlled Brownian-dynamics synthetic systems. In this setting, a known stochastic process generates equilibrium samples, and the simulator provides the corresponding score field for evaluation. This setting lets us test whether continuous normalizing flows and diffusion models learn both the equilibrium distribution of configurations and the local log-density gradients that govern overdamped dynamics.

For a configuration $x_t \in \mathbb{R}^{d\times N}$ with N beads in d spatial dimensions, we simulate the dynamics

$$dX_t = F(X_t)\,dt + \sqrt{2D}\,dW_t, \tag{1}$$

where D denotes the diffusion coefficient, W_t denotes standard Brownian motion, and $F(x)$ denotes the deterministic drift. For the current base Rouse simulations, $D = 0.25$. For the Rouse-chain experiments, adjacent harmonic springs generate the base drift,

$$F_i(x) = k_\xi \left(x_{i+1} - 2x_i + x_{i-1}\right), \tag{2}$$

with endpoint modifications. The implementation also supports nonideal terms, including Lennard-Jones interactions, excluded volume, confinement, hairpin contacts, and ring closure.

At equilibrium, the stationary-density score determines the drift:

$$F(x) = D\nabla_x \log p_{\mathsf{eq}}(x). \tag{3}$$

This relation gives a practical test for learned generative models. If a model learns $p_{\mathsf{eq}}(x)$, its learned score should align with the Brownian drift divided by D. For the current base Rouse setting, this relation gives the score target $F(x)/0.25 = 4F(x)$. The current experiments therefore learn equilibrium coordinate distributions from static snapshots, then compare generated samples and learned score fields with known Rouse-chain quantities.

II. HAMILTONIAN AND BOLTZMANN BACKGROUND

For atomistic molecular systems, we usually write the longer-term trajectory-inference framing in Hamiltonian form. Let $\vec{q}$ denote particle positions and $\vec{p}$ denote momenta. Hamilton's equations take the form

$$\frac{dq_{i,k}}{dt} = \frac{\partial H}{\partial p_{i,k}}, \tag{4}$$

$$\frac{dp_{i,k}}{dt} = -\frac{\partial H}{\partial q_{i,k}}, \tag{5}$$

where i indexes particles and k indexes spatial coordinates. For standard molecular dynamics, momenta satisfy

$$p_{i,k} = m_i \frac{dq_{i,k}}{dt}, \tag{6}$$

and we decompose the Hamiltonian into kinetic and potential energy:

$$H(\vec{q},\vec{p},t) = KE(\vec{p}) + PE(\vec{q}) = \sum_i \sum_k \frac{p_{i,k}^2}{2m_i} + V(\vec{q}). \tag{7}$$

At thermal equilibrium, the Boltzmann distribution gives the probability of a configuration:

$$P(\vec{q}) = \frac{1}{Z}\exp\left(-\frac{V(\vec{q})}{k_B T}\right), \tag{8}$$

with partition function

$$Z = \int \exp\left(-\frac{V(\vec{q})}{k_B T}\right) d\vec{q}. \tag{9}$$

Taking a gradient with respect to $q_{i,k}$ gives

$$\frac{dp_{i,k}}{dt} = -\frac{\partial H}{\partial q_{i,k}} = -\frac{\partial}{\partial q_{i,k}} V(\vec{q}) = k_B T \frac{\partial}{\partial q_{i,k}} \log P(\vec{q}). \tag{10}$$

Thus, a model that evaluates or differentiates the equilibrium log density can provide a force-like quantity. This relation bridges generative density modeling and trajectory inference. In the present codebase, we test this idea first in the overdamped Brownian setting, where the corresponding relation is $F(x) = D\nabla_x \log p_{\mathsf{eq}}(x)$, or equivalently $\nabla_x \log p_{\mathsf{eq}}(x) = F(x)/D$.

III. DATA

The training data contain centered bead configurations from simulated Rouse trajectories. The main data path loads HDF5 files containing arrays of shape

$$x \in \mathbb{R}^{d \times N \times B}, \tag{11}$$

where B denotes the number of frames. The current polymer configs simulate two-dimensional chains with $N = 32$ beads. For the base Rouse simulation, the SDE above generates trajectories with independent Brownian noise in every bead coordinate. The base simulation uses $D = 0.25$, unit bond length, and $k_\xi = 3.0$. The raw trajectories may include center-of-mass diffusion, but model training centers each frame so the models learn internal conformations rather than global translation.

IV. TIME CONVENTIONS

We use three time variables with different meanings. In the Brownian dynamics simulator, t represents physical simulation time. It indexes the stochastic trajectory that generates equilibrium snapshots. In the CNF, we use $\tau \in [0, 1]$ as a model pseudotime: $\tau = 0$ is the data distribution and $\tau = 1$ is the standard normal base distribution. Likelihood evaluation integrates from $\tau = 0$ to $\tau = 1$, while sampling integrates back from $\tau = 1$ to $\tau = 0$. In the diffusion model, $t \in [0, 1]$ represents noising time: $t = 0$ gives clean data and $t = 1$ gives approximately standard Gaussian noise. The reverse diffusion sampler starts at $t = 1$ and integrates to a small positive time, currently 10^{-4}, for numerical stability. These conventions match the implementation.

V. SYNTHETIC SYSTEMS

We focus on two controlled synthetic systems: the base Rouse chain and a hairpin-stabilized Rouse chain. The base system gives the simplest test of whether a model can learn the equilibrium distribution and score of a flexible Brownian polymer. The hairpin system adds local structure through attractive nonlocal contacts and repulsive excluded volume. We do not intend it as a protein model, but it gives a minimal analogue of a stable folded motif: the chain moves thermally, while a subset of long-range contacts stabilizes a preferred conformation.

Base Rouse Chain

The base Rouse model uses a quadratic bond potential between adjacent beads. In the notation of the score model, we write the dimensionless equilibrium potential as

$$U_{\mathsf{bond}}(x) = \frac{k_\xi}{2D} \sum_{i=2}^{N} \|x_i - x_{i-1}\|^2. \tag{12}$$

The corresponding equilibrium score satisfies

$$\nabla_x \log p_{\mathsf{eq}}(x) = -\nabla_x U_{\mathsf{bond}}(x), \tag{13}$$

and the Brownian drift satisfies $F(x) = D\nabla_x \log p_{\mathsf{eq}}(x)$. The base configuration uses $N = 32$ beads in two dimensions, $D = 0.25$, and $k_\xi = 3.0$. This setting disables all nonideal interactions. The base Rouse chain therefore provides a clean baseline because adjacent harmonic springs generate the equilibrium score entirely.

Nonideal Interactions

The simulator also supports several nonideal potentials. For nonbonded Lennard-Jones interactions, the potential has the form

$$U_{\mathsf{LJ}}(r) = 4\epsilon \left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6} \right], \tag{14}$$

with optional cutoff, softening, bonded-neighbor exclusion, and energy shifting at the cutoff. The force routine evaluates the corresponding radial derivative with softened squared distance $r^2 + \delta^2$ for numerical stability.

Excluded volume uses a purely repulsive power-law potential,

$$U_{\mathsf{EV}}(r) = \epsilon_{\mathsf{EV}} \left(\frac{\sigma_{\mathsf{EV}}}{r}\right)^{p}, \tag{15}$$

again with cutoff, softening, and bonded-neighbor exclusion. The current excluded-volume configuration inherits these defaults from the base file:

$$\epsilon_{\mathsf{EV}} = 0.03, \quad \sigma_{\mathsf{EV}} = 1.0, \quad p = 12, \quad \delta = 0.3, \quad r_{\mathsf{cut}} = 1.5. \tag{16}$$

This term prevents physically implausible bead overlap while preserving stochastic chain flexibility.

Hairpin-Stabilized Chain

The hairpin system starts from a two-strand initialization. For an even number of beads, the initializer places the first half of the chain on one strand and the second half on the opposing strand, with paired beads facing each other. The current hairpin configuration sets the initial strand separation to 0.95 and adds Gaussian initialization noise with standard deviation 0.02.

Attractive Lennard-Jones contacts between paired beads stabilize the hairpin:

$$i + j = N + 1, \qquad |i - j| \geq s_{\mathsf{min}}. \tag{17}$$

Only these designated long-range pairs receive the hairpin LJ interaction. The current configuration uses

$$\epsilon_{\mathsf{hp}} = 5.0, \quad \sigma_{\mathsf{hp}} = 0.9, \quad \delta_{\mathsf{hp}} = 0.3, \\ r_{\mathsf{cut}} = 3.0, \quad s_{\mathsf{min}} = 2. \tag{18}$$

The hairpin configuration also turns on excluded volume and centered confinement. The confinement term uses the potential

$$U_{\mathsf{conf}}(x) = c \sum_{i=1}^{N} \|x_i - \bar{x}\|^2, \qquad c = 0.02, \tag{19}$$

where $\bar{x}$ denotes the chain center of mass. Together, these terms create a stable but thermally fluctuating structure. This synthetic example comes closest to the protein motivation: the model must learn a broad polymer ensemble and a distribution shaped by stabilizing nonlocal contacts.

VI. CONTINUOUS NORMALIZING FLOW

The CNF model learns an ODE that maps data coordinates to a standard normal base distribution. Let $z(\tau)$ denote the CNF state at pseudotime $\tau \in [0, 1]$:

$$\frac{dz(\tau)}{d\tau} = f_\theta(z(\tau), \tau). \tag{20}$$

Here, τ is a generative-model coordinate, not physical Brownian time. The CNF integrates from data to noise for likelihood evaluation and from noise to data for sampling.

The log-density evolves according to the instantaneous change-of-variables formula:

$$\frac{d}{d\tau} \log p(z(\tau)) = -\,\mathsf{Tr}\left(\frac{\partial f_\theta}{\partial z}\right). \tag{21}$$

Equivalently, the accumulated log-density change satisfies

$$\log P(z(\tau)) - \log P(z(0)) = \Delta \log P_z(\tau) \\ = \int_0^\tau -\,\mathsf{Tr}\left(\frac{\partial f_\theta}{\partial z(\lambda)}\right) d\lambda. \tag{22}$$

For a training sample x, the model solves

$$z(0) = x, \tag{23}$$
$$z(1) = \mathsf{ODESolve}(z(0), f_\theta, 0, 1). \tag{24}$$

The model then computes the log likelihood as

$$\log p_\theta(x) = \log \mathcal{N}(z(1); 0, I) - \Delta \log p, \tag{25}$$

where the ODE solve accumulates $\Delta \log p$. We can write the same calculation as the augmented ODE system

$$\begin{bmatrix} z(1) \\ \Delta \log P_z(1) \end{bmatrix} = \begin{bmatrix} z(0) = x \\ \Delta \log P_z(0) = 0 \end{bmatrix} \\ + \int_0^1 \begin{bmatrix} f_\theta(z(\tau), \tau) \\ -\,\mathsf{Tr}\left(\frac{\partial f_\theta}{\partial z(\tau)}\right) \end{bmatrix} d\tau. \tag{26}$$

The training objective minimizes negative log likelihood:

$$\mathcal{L}_{\mathsf{CNF}}(\theta) = -\mathbb{E}_{x \sim p_{\mathsf{data}}} \left[\log p_\theta(x)\right]. \tag{27}$$

The current polymer CNF uses an EGNN-style vector field. The network builds pairwise messages from node embeddings, sequence separation $i - j$, squared pair distances $\|x_i - x_j\|^2$, and pseudotime τ. It forms coordinate updates from relative displacements, so the vector field remains translation equivariant after centering. The main EGNN CNF configuration estimates the divergence term with a Hutchinson JVP trace estimator rather than an exact trace. Sampling starts from Gaussian noise and integrates the same CNF dynamics backward from $\tau = 1$ to $\tau = 0$.

Adjoint Sensitivity Equations

The original CNF formulation can compute parameter gradients with an adjoint ODE. For the scalar objective

$$\mathcal{L} = \log P(z(1)) - \Delta \log P_z(1), \tag{28}$$

define the adjoint state

$$a(\tau) = \begin{bmatrix} a_z(\tau) \\ a_\Delta(\tau) \end{bmatrix} = \begin{bmatrix} \partial \mathcal{L}/\partial z \\ \partial \mathcal{L}/\partial(\Delta \log P_z) \end{bmatrix}. \tag{29}$$

At $\tau = 1$,

$$a(1) = \begin{bmatrix} \nabla_z \log P(z(1)) \\ -1 \end{bmatrix}. \tag{30}$$

The adjoint dynamics satisfy

$$\frac{da(\tau)}{d\tau} = -\begin{bmatrix} \frac{\partial f_\theta}{\partial z} & 0 \\ -\nabla_z\,\mathsf{Tr}\left(\frac{\partial f_\theta}{\partial z}\right) & 0 \end{bmatrix}^T a(\tau), \tag{31}$$

$$\frac{d}{d\tau}\left(\frac{\partial \mathcal{L}}{\partial \theta}\right) = a(\tau)^T \begin{bmatrix} \frac{\partial f_\theta}{\partial \theta} \\ -\nabla_\theta\,\mathsf{Tr}\left(\frac{\partial f_\theta}{\partial z}\right) \end{bmatrix}. \tag{32}$$

The Julia code delegates differentiable ODE solves and adjoint machinery to the SciML stack rather than manually coding these equations in the experiment loop.

VII. VARIANCE-PRESERVING DIFFUSION MODEL

The diffusion baseline learns the score field directly. The implementation defines a continuous-time variance-preserving (VP) forward SDE

$$dx_t = -\frac{1}{2}\beta(t)x_t\,dt + \sqrt{\beta(t)}\,dw_t, \tag{33}$$

with a linear noise-rate schedule

$$\beta(t) = \beta_{\text{min}} + t(\beta_{\text{max}} - \beta_{\text{min}}), \quad t \in [0,1], \tag{34}$$

with $\beta_{\text{min}} = 0.1$ and $\beta_{\text{max}} = 20$ in the code. The marginal forward process has the form

$$x_t = \mu(t)x_0 + \sigma(t)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{35}$$

where

$$\mu(t) = \exp\left[-\frac{1}{4}(\beta_{\text{max}} - \beta_{\text{min}})t^2 - \frac{1}{2}\beta_{\text{min}}t\right], \tag{36}$$

$$\sigma^2(t) = 1 - \exp\left[-\frac{1}{2}(\beta_{\text{max}} - \beta_{\text{min}})t^2 - \beta_{\text{min}}t\right]. \tag{37}$$

The code centers both x_0 and the sampled noise, so the diffusion process stays in the centered coordinate subspace that training uses. For numerical stability, it clamps training times below by 10^{-5} and floors $\sigma(t)$ at the corresponding minimum positive value before forming the score target.

We train the score network $s_\theta(x_t, t)$ to estimate

$$\nabla_{x_t} \log p_{0t}(x_t|x_0) = -\frac{\epsilon}{\sigma(t)}. \tag{38}$$

We train with the weighted denoising score-matching objective

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{t,x_0,\epsilon}\left[\sigma^2(t)\left\|s_\theta(x_t,t) + \frac{\epsilon}{\sigma(t)}\right\|_2^2\right]. \tag{39}$$

The diffusion score model also uses an EGNN-style architecture. It keeps learned node embeddings, uses sequence-distance and radial pair features, and updates coordinates through pairwise relative displacements. The code centers the output and interprets it directly as $\nabla_x \log p_t(x)$.

Generation follows the reverse-time VP SDE. For this linear schedule, the reverse coefficients satisfy

$$f(t) = -\frac{1}{2}\beta(t), \quad g(t)^2 = \beta(t). \tag{40}$$

The Euler-Maruyama sampler starts from centered Gaussian noise at $t = 1$ and steps backward to a small positive time:

$$x_{t-\Delta t} = x_t - \left[f(t)x_t - g(t)^2 s_\theta(x_t,t)\right]\Delta t + g(t)\sqrt{\Delta t}\,z, \qquad z \sim \mathcal{N}(0,I). \tag{41}$$

By default, the sampler integrates from $t = 1$ to $t = 10^{-4}$. For numerical stability, it clips sample score norms to 64, recenters each state, checks for non-finite samples, and omits the final noise draw on the last step. During diffusion training, the optimizer also clips parameter gradients elementwise to magnitude 1.

VIII. EVALUATION

The experiments report sample-level statistics and score-field diagnostics. For generated samples, they use the mean absolute error between average pairwise distances in generated configurations and training configurations as the main structural metric. The code also records the fraction of generated values and samples that remain finite.

For Rouse and polymer Langevin data, the simulator provides analytic force targets. We obtain the CNF score by differentiating the learned log likelihood,

$$\nabla_x \log p_\theta(x), \tag{42}$$

while we take the diffusion score from the network output at a small time value near zero. The evaluation scripts compare these scores with the known Brownian score target $F(x)/D$ and apply normalization corrections when the data normalizer runs. This benchmark goes beyond visual sample comparison: the model should generate plausible chain conformations and recover the local equilibrium score implied by the Brownian dynamics.

IX. JULIA AND SCIML IMPLEMENTATION

We build the implementation around the SciML ecosystem. This choice does more than provide convenience: the project needs stochastic simulation, neural ODE likelihoods, differentiable ODE solves, GPU-compatible array code, and several trace-estimation strategies in the same code path. Julia makes these pieces composable without translating between separate simulation, autodiff, and machine-learning frameworks.

Brownian Data Generation with StochasticDiffEq

The Rouse data generator constructs the simulation directly as an SDE problem. The simulator constructs

$$dX_t = F(X_t)\,dt + \sqrt{2D}\,dW_t \tag{43}$$

as a `SDEProblem`; the drift function calls the same polymer force routine that the evaluation code later uses for score targets. The raw trajectory script calls `StochasticDiffEq` solvers through a configurable solver interface. The available choices include Euler-Maruyama, Euler-Heun, RKMil, SOSRA, and SOSRA2.

This lets the same model family generate inexpensive baseline trajectories or more stable stochastic trajectories by changing a YAML configuration rather than rewriting simulation code.

The learning experiments then reuse the saved trajectories through HDF5 loaders. This structure separates stochastic simulation from generative-model training while keeping both stages inside Julia.

CNF Likelihoods as SciML ODE Problems

The CNF implementation defines an `ODEProblem` whose `ComponentArray` state contains both coordinates and accumulated log-density change:

$$u(\tau) = (x(\tau), \Delta \log p(\tau)) \,. \tag{44}$$

The ODE right-hand side returns

$$\frac{d}{d\tau}\begin{bmatrix} x(\tau) \\ \Delta \log p(\tau) \end{bmatrix} = \begin{bmatrix} f_\theta(x(\tau), \tau) \\ -\operatorname{Tr}\left(\partial f_\theta / \partial x\right) \end{bmatrix}. \tag{45}$$

The code solves the same problem definition forward for likelihood evaluation and backward for sample generation. The configs select the solver from the SciML ODE stack; current configs use adaptive `Tsit5` and support fixed-step `RK4`. For training, the solve uses `InterpolatingAdjoint(autojacvec=ZygoteVJP())`, so gradients propagate through the ODE solve while remaining compatible with the neural vector field and the trace estimator.

This design keeps the generative model close to its mathematical definition. The code avoids a separate CNF framework: the likelihood model is an ODE with a structured state, a vector field, a trace estimator, and a SciML adjoint.

Hutchinson JVP Trace Estimator

The main EGNN CNF configuration uses a Hutchinson trace estimator with a Jacobian-vector product (JVP). For a vector field $f_\theta(x, \tau)$, the Hutchinson identity estimates the divergence as

$$\operatorname{Tr}\left(\frac{\partial f_\theta}{\partial x}\right) = \mathbb{E}_v\left[v^T \frac{\partial f_\theta}{\partial x} v\right], \tag{46}$$

where v denotes a Rademacher probe. In the SciML implementation, the code samples this probe when it constructs the `ODEProblem` for a batch, captures it in the ODE right-hand-side closure, and reuses that fixed probe for every RHS evaluation in the adaptive solve. The augmented RHS returns both the coordinate velocity $f_\theta(x, \tau)$ and the log-density derivative $-\widehat{\operatorname{Tr}}$, so the SciML solver integrates the stochastic trace estimate as part of the same state as the coordinates. The implementation samples one probe per batch and computes

$$\widehat{\operatorname{Tr}} = \sum_{i,j} v_{i,j} \left[J_{f_\theta}(x, \tau) v\right]_{i,j} . \tag{47}$$

The implementation propagates the EGNN JVP explicitly through the full vector field, not only through the final coordinate update. For the current multi-layer EGNN, each layer maintains node features h_i^ℓ and their directional derivatives dh_i^ℓ with respect to the coordinate probe v. It initializes $dh_i^0 = 0$ because the learned node embeddings do not depend on the coordinates, then propagates derivatives through pairwise geometry, edge MLPs, coordinate aggregation, message aggregation, and node updates. For example, if $r_{ij} = x_i - x_j$, then the JVP carries the directional derivatives

$$dr_{ij} = v_i - v_j, \qquad d\|r_{ij}\|^2 = 2 r_{ij}^T dr_{ij}. \tag{48}$$

The edge input at layer ℓ contains

$$\left(h_i^\ell, h_j^\ell, i - j, \|r_{ij}\|^2, \tau\right), \tag{49}$$

while its JVP input contains

$$\left(dh_i^\ell, dh_j^\ell, 0, 2 r_{ij}^T dr_{ij}, 0\right). \tag{50}$$

The helper `forward_mlp_generic_jvp` then propagates the value and derivative together through each affine layer and `tanh` nonlinearity. The edge network outputs a coordinate weight w_{ij}^ℓ and node message m_{ij}^ℓ, along with derivatives dw_{ij}^ℓ and dm_{ij}^ℓ. The coordinate JVP for one EGNN layer follows the product rule:

$$d\Delta x_i^\ell = \frac{1}{N} \sum_j M_{ij} \left(dw_{ij}^\ell r_{ij} + w_{ij}^\ell dr_{ij}\right), \tag{51}$$

where M_{ij} masks self-edges. The node-feature JVP uses the derivative of the aggregated messages,

$$d\bar{m}_i^\ell = \frac{1}{N} \sum_j M_{ij} dm_{ij}^\ell, \tag{52}$$

passes $(dh_i^\ell, d\bar{m}_i^\ell, 0)$ through the node MLP JVP, and updates $dh_i^{\ell+1}$ together with the primal node features. After all layers, the implementation averages both accumulated coordinate updates and accumulated directional derivatives by the number of EGNN layers. Thus the analytic path returns both $f_\theta(x, \tau)$ and the full $J_{f_\theta}(x, \tau)v$ in one structured forward pass. This approach avoids materializing the full Jacobian over all bead coordinates. For a 32-bead chain in two dimensions, the full Jacobian already forms a dense object over 64 coordinate variables per sample; for larger systems, exact traces become the wrong abstraction. The JVP estimator keeps the CNF likelihood scalable while preserving the same ODE interface.

The code also keeps alternative trace modes and provides an exact Zygote-based divergence for smaller

checks. It provides a finite-difference Hutchinson mode, `hutchinson_fd`, which approximates the same JVP by

$$J_{f_\theta}(x,\tau)v \approx \frac{f_\theta(x+\epsilon v,\tau) - f_\theta(x-\epsilon v,\tau)}{2\epsilon}. \tag{53}$$

This finite-difference path gives a useful diagnostic fallback because it tests the same trace estimator without relying on the analytic JVP implementation. The production path uses `hutchinson_jvp` because it runs faster and remains analytic: it avoids two extra vector-field evaluations per trace estimate, avoids finite-difference step-size error, and remains differentiable through the exact directional-derivative computation. Training still uses Zygote for gradients through the CNF loss and SciML adjoint solve, but Zygote does not derive this EGNN JVP automatically; the model code implements the JVP explicitly.

Autodiff and Stability

Both CNF and diffusion training use Zygote for reverse-mode gradients. Several small custom adjoints remove avoidable AD overhead for common tensor operations such as batched node embeddings, row concatenation, and centering. The diffusion code also uses `Zygote.ignore` and `dropgrad` where fixed geometric inputs, masks, sampled noise, or numerical guards should not enter the trainable derivative path.

The result keeps stochastic simulation, adaptive ODE solving, adjoint differentiation, neural network training, and GPU execution in one language and one type system. This gives Julia its main advantage for this project: the mathematical objects in the paper map directly to executable SciML problems, and the engineering choices needed for stability remain visible in the code rather than hidden behind a separate modeling layer.

X. TRAJECTORY-INFERENCE ALGORITHM

We can summarize the full long-term BoltzFlow trajectory-inference idea from the earlier proposal as follows. First, obtain equilibrium snapshots

$$\{\vec{q}_1,\ldots,\vec{q}_n\} \sim P(\vec{q}). \tag{54}$$

Then train a CNF to model $P(\vec{q})$ and evaluate $\log P(\vec{q})$. For a trajectory initialized at $\vec{q}(t_0)$, sample momenta from the Maxwell-Boltzmann distribution,

$$p_{i,k}(t_0) \sim \mathcal{N}\left(0, m_i k_B T\right). \tag{55}$$

At each physical timestep, compute the CNF log density and its gradient:

$$\nabla_{\vec{q}} \log P(\vec{q}(t)). \tag{56}$$

Using the Boltzmann relation,

$$-\nabla_{\vec{q}} V(\vec{q}) = k_B T \nabla_{\vec{q}} \log P(\vec{q}), \tag{57}$$

update the momenta and positions with Hamilton's equations:

$$\frac{d\vec{p}}{dt} = k_B T \nabla_{\vec{q}} \log P(\vec{q}), \tag{58}$$

$$\frac{d\vec{q}}{dt} = \frac{\partial H}{\partial \vec{p}}. \tag{59}$$

This Hamiltonian trajectory-inference loop motivates the protein-facing direction. The current implementation does not yet execute this loop for atomistic proteins; instead, it validates the score-learning components on Brownian/Rouse systems where we know the target force field.

XI. CURRENT SCOPE

We should describe the current codebase as a Brownian-dynamics and polymer-equilibrium study. Protein dynamics remain a longer-term motivation, but the implemented system does not yet infer atomistic protein trajectories from static structural ensembles. The code supports a narrower practical claim: we can train equivariant CNF and diffusion models on centered equilibrium snapshots from Rouse-style Brownian simulations, sample them as generative models, and evaluate them against known equilibrium score fields.

Appendix A: Derivations

Throughout the appendix, the *score* of a density $p(x)$ means the gradient of its log density, $\nabla_x \log p(x)$. It points in the direction where the log probability increases most rapidly. For stochastic differential equations, dW_t denotes a Brownian increment with mean zero and variance dt, so terms proportional to dW_t create fluctuations of size $\sqrt{dt}$ while drift terms proportional to dt create deterministic motion.

1. Boltzmann Score and Conservative Forces

At equilibrium, an atomistic configuration $\vec{q}$ with potential energy $V(\vec{q})$ follows the Boltzmann density

$$\begin{aligned} P(\vec{q}) &= \frac{1}{Z}\exp\left(-\frac{V(\vec{q})}{k_B T}\right), \\ Z &= \int \exp\left(-\frac{V(\vec{q})}{k_B T}\right) d\vec{q}. \end{aligned} \tag{A1}$$

This expression assumes the canonical ensemble at fixed temperature after marginalizing over momenta. It

also assumes that the equilibrium positional density depends on $\vec{q}$ only through the potential energy. Taking the logarithm gives

$$\log P(\vec{q}) = -\frac{V(\vec{q})}{k_B T} - \log Z. \tag{A2}$$

The partition function Z does not depend on $\vec{q}$, so differentiating with respect to the coordinates yields

$$\nabla_{\vec{q}} \log P(\vec{q}) = -\frac{1}{k_B T} \nabla_{\vec{q}} V(\vec{q}). \tag{A3}$$

Rearranging gives the force

$$-\nabla_{\vec{q}} V(\vec{q}) = k_B T \nabla_{\vec{q}} \log P(\vec{q}). \tag{A4}$$

Thus any differentiable model of the equilibrium log density provides a conservative force estimate when multiplied by $k_B T$. The resulting force field is conservative because it derives from the scalar potential $-k_B T \log P(\vec{q})$. This statement motivates using a learned density model to drive Hamiltonian dynamics.

2. Overdamped Brownian Score Relation

The Brownian systems in the code follow the Itô SDE

$$dX_t = F(X_t)\, dt + \sqrt{2D}\, dW_t. \tag{A5}$$

The corresponding Fokker-Planck equation for the density $p(x,t)$ is

$$\frac{\partial p}{\partial t} = -\nabla_x \cdot (F(x) p(x,t)) + D \nabla_x^2 p(x,t). \tag{A6}$$

At stationarity, $\partial p / \partial t = 0$. Writing the probability current as

$$J(x) = F(x) p_{\mathsf{eq}}(x) - D \nabla_x p_{\mathsf{eq}}(x), \tag{A7}$$

Detailed balance sets $J(x) = 0$ at equilibrium. This condition is stronger than stationarity alone: a nonequilibrium steady state can satisfy $\nabla_x \cdot J(x) = 0$ while still having a nonzero circulating current. The Brownian/Rouse score target assumes the reversible equilibrium case, so the current vanishes pointwise. Therefore

$$F(x) p_{\mathsf{eq}}(x) = D \nabla_x p_{\mathsf{eq}}(x). \tag{A8}$$

Dividing by $p_{\mathsf{eq}}(x)$ gives

$$F(x) = D \frac{\nabla_x p_{\mathsf{eq}}(x)}{p_{\mathsf{eq}}(x)} = D \nabla_x \log p_{\mathsf{eq}}(x). \tag{A9}$$

Thus the equilibrium score target for the Brownian simulations is

$$\nabla_x \log p_{\mathsf{eq}}(x) = \frac{F(x)}{D}. \tag{A10}$$

For the base Rouse experiments, $D = 0.25$, so the score target becomes $4F(x)$.

3. Rouse-Chain Score

For the base Rouse chain, the dimensionless equilibrium potential is

$$U_{\mathsf{bond}}(x) = \frac{k_\xi}{2D} \sum_{i=2}^{N} \|x_i - x_{i-1}\|^2. \tag{A11}$$

The equilibrium density obeys

$$p_{\mathsf{eq}}(x) \propto \exp\left[-U_{\mathsf{bond}}(x)\right], \tag{A12}$$

so

$$\nabla_{x_i} \log p_{\mathsf{eq}}(x) = -\nabla_{x_i} U_{\mathsf{bond}}(x). \tag{A13}$$

For an interior bead, $1 < i < N$, only the two adjacent bonds contribute:

$$\begin{aligned} \nabla_{x_i} U_{\mathsf{bond}}(x) &= \frac{k_\xi}{2D} \nabla_{x_i} \left(\|x_i - x_{i-1}\|^2 + \|x_{i+1} - x_i\|^2 \right) \\ &= \frac{k_\xi}{D} \left(2x_i - x_{i-1} - x_{i+1}\right). \end{aligned} \tag{A14}$$

Therefore

$$\nabla_{x_i} \log p_{\mathsf{eq}}(x) = \frac{k_\xi}{D} \left(x_{i+1} - 2x_i + x_{i-1}\right), \tag{A15}$$

and the Brownian drift becomes

$$F_i(x) = D \nabla_{x_i} \log p_{\mathsf{eq}}(x) = k_\xi \left(x_{i+1} - 2x_i + x_{i-1}\right), \tag{A16}$$

with one-sided endpoint modifications. Explicitly, the endpoints satisfy

$$F_1(x) = k_\xi (x_2 - x_1), \tag{A17}$$

$$F_N(x) = k_\xi (x_{N-1} - x_N). \tag{A18}$$

The uncentered Rouse potential is invariant to global translations, so the equilibrium density has a translation zero mode. The experiments remove this mode by centering every configuration before training and evaluation; the models therefore learn the internal conformational distribution rather than center-of-mass diffusion.

The nonideal hairpin experiment adds pair potentials to the same score. For a pair (i,j), write

$$r_{ij} = x_i - x_j, \qquad \rho_{ij}^2 = \|r_{ij}\|^2 + \delta^2, \tag{A19}$$

where δ is the softening length used in the simulator. Cutoffs and bonded-neighbor exclusions select which pairs contribute. For any active pair potential $U_{ij}(\rho_{ij})$, the pair score on bead i is

$$s_{i \leftarrow j}(x) = -\nabla_{x_i} U_{ij}, \qquad s_{j \leftarrow i}(x) = -s_{i \leftarrow j}(x). \tag{A20}$$

Excluded volume uses the repulsive power-law potential

$$U_{\mathsf{EV},ij} = \epsilon_{\mathsf{EV}} \left(\frac{\sigma_{\mathsf{EV}}}{\rho_{ij}} \right)^p. \tag{A21}$$

Differentiating gives the score contribution

$$s^{\mathsf{EV}}_{i\leftarrow j} = \epsilon_{\mathsf{EV}} p \frac{\sigma^p_{\mathsf{EV}}}{\rho^{p+2}_{ij}} r_{ij}. \tag{A22}$$

This vector points away from bead j, so the term penalizes bead overlap. The hairpin run enables this term with the base defaults

$$\begin{aligned} \epsilon_{\mathsf{EV}} = 0.03, \quad \sigma_{\mathsf{EV}} = 1.0, \quad p = 12, \\ \delta = 0.3, \quad r_{\mathsf{cut}} = 1.5. \end{aligned} \tag{A23}$$

and excludes bonded neighbors.

The Lennard-Jones potential used for attractive hairpin contacts is

$$U_{\mathsf{LJ},ij} = 4\epsilon \left[\left(\frac{\sigma}{\rho_{ij}} \right)^{12} - \left(\frac{\sigma}{\rho_{ij}} \right)^{6} \right]. \tag{A24}$$

Define

$$a_{ij} = \frac{\sigma^2}{\rho^2_{ij}}, \qquad a^3_{ij} = \left(\frac{\sigma}{\rho_{ij}} \right)^6, \qquad a^6_{ij} = \left(\frac{\sigma}{\rho_{ij}} \right)^{12}. \tag{A25}$$

Then the LJ score contribution is

$$s^{\mathsf{LJ}}_{i\leftarrow j} = 24\epsilon \frac{1}{\rho^2_{ij}} \left(2a^6_{ij} - a^3_{ij} \right) r_{ij}. \tag{A26}$$

The first term is strongly repulsive at short distance, while the second term creates attraction once the beads are outside the repulsive core. The code optionally subtracts the LJ energy value at the cutoff; this shift changes the potential by a constant inside the cutoff and therefore does not change the score. The hairpin experiment applies this LJ term only to paired beads

$$\mathcal{H} = \{(i,j) : i + j = N + 1,\ |i - j| \geq s_{\mathsf{min}}\}, \tag{A27}$$

with

$$\begin{aligned} \epsilon_{\mathsf{hp}} = 5.0, \quad \sigma_{\mathsf{hp}} = 0.9, \quad \delta_{\mathsf{hp}} = 0.3, \\ r_{\mathsf{cut}} = 3.0, \quad s_{\mathsf{min}} = 2. \end{aligned} \tag{A28}$$

For the hairpin system, the full equilibrium score is the sum of the bonded Rouse score, all active excluded-volume pair scores, the active hairpin LJ pair scores, and the centered confinement score. This combined expression is the target produced by `polymer_langevin_score!`; multiplying it by D gives the Brownian drift used in simulation.

4. Continuous Normalizing Flow Log Density

The CNF defines an invertible flow by the ODE

$$\frac{dz(\tau)}{d\tau} = f_\theta(z(\tau), \tau). \tag{A29}$$

Let $p(z(\tau), \tau)$ denote the density of states at pseudotime τ. Conservation of probability under deterministic transport gives the continuity equation

$$\frac{\partial p}{\partial \tau} + \nabla_z \cdot (p f_\theta) = 0. \tag{A30}$$

To expand the divergence term, apply the product rule component by component. If z_m denotes one coordinate of z, then

$$\begin{aligned} \nabla_z \cdot (p f_\theta) &= \sum_m \frac{\partial}{\partial z_m} (p f_{\theta,m}) \\ &= \sum_m \left[f_{\theta,m} \frac{\partial p}{\partial z_m} + p \frac{\partial f_{\theta,m}}{\partial z_m} \right] \\ &= f_\theta \cdot \nabla_z p + p\, \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z} \right). \end{aligned} \tag{A31}$$

The first term says that the vector field carries density along its direction of motion. The second term measures local expansion or compression of the flow through the divergence $\nabla_z \cdot f_\theta = \mathsf{Tr}(\partial f_\theta / \partial z)$. Substituting this product-rule expansion gives

$$\frac{\partial p}{\partial \tau} + f_\theta \cdot \nabla_z p + p\, \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z} \right) = 0. \tag{A32}$$

Along a trajectory $z(\tau)$, the density depends on both the current position and pseudotime. The chain rule gives

$$\frac{d}{d\tau} p(z(\tau), \tau) = \frac{\partial p}{\partial \tau} + \frac{dz}{d\tau} \cdot \nabla_z p. \tag{A33}$$

Substituting the CNF ODE, $dz/d\tau = f_\theta(z(\tau), \tau)$, yields

$$\frac{d}{d\tau} p(z(\tau), \tau) = \frac{\partial p}{\partial \tau} + f_\theta \cdot \nabla_z p. \tag{A34}$$

The first two terms in the expanded continuity equation are therefore exactly this total derivative along the flow. Combining the expanded continuity equation with the chain-rule expression gives

$$\begin{aligned} \frac{d}{d\tau} p(z(\tau), \tau) + p(z(\tau), \tau)\, \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z} \right) = 0, \\ \frac{d}{d\tau} p(z(\tau), \tau) = -p(z(\tau), \tau)\, \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z} \right). \end{aligned} \tag{A35}$$

Dividing by p gives the instantaneous change-of-variables formula

$$\frac{d}{d\tau} \log p(z(\tau), \tau) = -\, \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z} \right). \tag{A36}$$

Integrating from $\tau = 0$ to $\tau = 1$ gives

$$\log p(z(1), 1) - \log p(z(0), 0) = -\int_0^1 \mathsf{Tr} \left(\frac{\partial f_\theta}{\partial z(\tau)} \right) d\tau. \tag{A37}$$

With $z(0) = x$ and standard normal base density at $z(1)$,

$$\log p_\theta(x) = \log \mathcal{N}(z(1); 0, I) + \int_0^1 \mathrm{Tr}\left(\frac{\partial f_\theta}{\partial z(\tau)}\right) d\tau. \tag{A38}$$

The augmented CNF state stores the integrated log-density change rather than the absolute density. Here $\Delta \log p(\tau) = \log p(z(\tau), \tau) - \log p(z(0), 0)$, so differentiating removes the constant initial term. Using the instantaneous change-of-variables result above, the implementation therefore accumulates

$$\begin{aligned} &\frac{d}{d\tau}\Delta \log p = -\mathrm{Tr}\left(\frac{\partial f_\theta}{\partial z}\right), \\ &\Delta \log p(0) = 0, \end{aligned} \tag{A39}$$

and evaluates

$$\begin{aligned} \log p_\theta(x) = \log \mathcal{N}(z(1); 0, I) \\ - \Delta \log p(1). \end{aligned} \tag{A40}$$

5. Hutchinson Trace Estimator

Let A denote the Jacobian matrix $\partial f_\theta / \partial x$. The CNF log-density equation needs the divergence of the vector field,

$$\nabla_x \cdot f_\theta(x, \tau) = \mathrm{Tr}\left(\frac{\partial f_\theta}{\partial x}\right). \tag{A41}$$

This trace is the sum of all same-coordinate sensitivities: how much each output coordinate changes when its matching input coordinate changes. For a large bead system, forming the full Jacobian is wasteful because the CNF only needs this one scalar trace. Hutchinson's estimator recovers that scalar from a random probe and one Jacobian-vector product. For a Rademacher probe v with independent entries satisfying

$$\mathbb{E}[v_i] = 0, \qquad \mathbb{E}[v_i v_j] = \delta_{ij}, \tag{A42}$$

we have

$$\begin{aligned} \mathbb{E}\left[v^T A v\right] &= \mathbb{E}\left[\sum_i \sum_j v_i A_{ij} v_j\right] \\ &= \sum_i \sum_j A_{ij} \mathbb{E}\left[v_i v_j\right] \\ &= \sum_i A_{ii} = \mathrm{Tr}(A). \end{aligned} \tag{A43}$$

Thus

$$v^T \frac{\partial f_\theta}{\partial x} v \tag{A44}$$

gives an unbiased stochastic estimate of the divergence. Here "unbiased" means the estimate may vary for one random probe, but its average over probes equals the exact trace. The same calculation can be read component by component. The matrix-vector product Av has entries

$$(Av)_i = \sum_j A_{ij} v_j, \tag{A45}$$

so the scalar Hutchinson estimate is

$$v^T A v = \sum_i v_i (Av)_i = \sum_i \sum_j A_{ij} v_i v_j. \tag{A46}$$

If $i \neq j$, then v_i and v_j are independent signs, so $\mathbb{E}[v_i v_j] = 0$. If $i = j$, then $v_i^2 = 1$. Thus the random probe removes the off-diagonal terms in expectation and keeps the diagonal terms:

$$\mathbb{E}[v^T A v] = \sum_i A_{ii}. \tag{A47}$$

For bead coordinates, the index i in this argument represents one bead-coordinate pair (a, k). The trace therefore sums each matching response

$$\mathrm{Tr}(A) = \sum_{a,k} \frac{\partial f_{\theta,a,k}}{\partial x_{a,k}}. \tag{A48}$$

In the SciML implementation, `_cnf_problem` samples one $\{-1, +1\}$ probe with the same shape as the batch coordinates before it constructs the `ODEProblem`. The ODE right-hand side keeps that probe fixed for the whole solve. Adaptive substeps, rejected steps, and adjoint replay therefore evaluate the same trace direction. If the code drew a new probe at every RHS call, the ODE would see extra time-dependent noise rather than one well-defined stochastic trace estimate. For each RHS call, the augmented derivative has the schematic form

$$\frac{d}{d\tau}\begin{bmatrix} x \\ \Delta \log p \end{bmatrix} = \begin{bmatrix} f_\theta(x, \tau) \\ -v^T J_{f_\theta}(x, \tau) v \end{bmatrix}. \tag{A49}$$

The code computes the matrix-vector product

$$J_{f_\theta}(x, \tau) v = \frac{d}{d\alpha} f_\theta(x + \alpha v, \tau)\Big|_{\alpha=0} \tag{A50}$$

analytically for the EGNN, then forms

$$\widehat{\mathrm{Tr}} = \sum_{i,k} v_{i,k} \left[J_{f_\theta}(x, \tau) v\right]_{i,k}, \tag{A51}$$

where i indexes beads and k indexes spatial coordinates. During training, the loss differentiates through this stochastic trace estimate with respect to the network parameters.

6. Full EGNN JVP Recursion

This subsection writes the analytic JVP that `_egnn_velocity_jvp` implements. At layer ℓ, the

EGNN stores node features h_i^ℓ and coordinate state x_i. The JVP in this subsection differentiates the vector field with respect to coordinates in the probe direction v. It does not compute a parameter derivative; Zygote still differentiates the final CNF loss with respect to the trainable parameters after the code computes this coordinate JVP. The notation $d(\cdot)$ means directional derivative along the perturbed coordinates $x + \alpha v$:

$$dQ = \frac{d}{d\alpha} Q(x + \alpha v)\Big|_{\alpha=0}. \tag{A52}$$

For example, $dx_i = v_i$ because the perturbed coordinate is $x_i + \alpha v_i$. The goal is to propagate these $d(\cdot)$ quantities through the same operations that compute the EGNN velocity. At the end, the propagated derivative is exactly $J_{f_\theta}(x, \tau)v$. For a coordinate probe v_i, the JVP stores

$$dx_i = v_i, \qquad dh_i^0 = 0. \tag{A53}$$

The initial node features have zero directional derivative because they are learned bead embeddings and do not change when the coordinates move. The pairwise relative displacement and squared distance satisfy

$$r_{ij} = x_i - x_j, \qquad d_{ij} = \|r_{ij}\|^2, \tag{A54}$$

with directional derivatives

$$\begin{aligned} dr_{ij} &= dx_i - dx_j = v_i - v_j, \\ dd_{ij} &= d(r_{ij}^T r_{ij}) = 2 r_{ij}^T dr_{ij}. \end{aligned} \tag{A55}$$

Thus the JVP first tracks how each pair displacement and squared pair distance changes under the probe. The edge network receives

$$e_{ij}^\ell = \left(h_i^\ell, h_j^\ell, i - j, d_{ij}, \tau\right) \tag{A56}$$

and directional input

$$de_{ij}^\ell = \left(dh_i^\ell, dh_j^\ell, 0, dd_{ij}, 0\right). \tag{A57}$$

The zero entries appear because sequence separation $i - j$ and pseudotime τ do not change when the coordinate probe moves x. The MLP JVP applies the chain rule layer by layer. For an affine transformation followed by tanh,

$$a = Wy + b, \qquad da = W\, dy, \tag{A58}$$

$$y^+ = \tanh(a), \quad dy^+ = \left(1 - \tanh^2(a)\right) \odot da. \tag{A59}$$

This is the same rule used in ordinary calculus: differentiate each operation and pass the derivative to the next operation. The final layer omits the nonlinearity. The edge MLP therefore returns

$$(w_{ij}^\ell, m_{ij}^\ell), \qquad (dw_{ij}^\ell, dm_{ij}^\ell), \tag{A60}$$

where w_{ij}^ℓ denotes the scalar coordinate weight and m_{ij}^ℓ denotes the node message. The scalar w_{ij}^ℓ controls how strongly bead j moves bead i along the relative displacement r_{ij}. The message m_{ij}^ℓ updates the hidden node features. With self-edge mask M_{ij}, the primal coordinate update is

$$\Delta x_i^\ell = \frac{1}{N} \sum_j M_{ij} w_{ij}^\ell r_{ij}. \tag{A61}$$

The JVP follows from the product rule:

$$d\Delta x_i^\ell = \frac{1}{N} \sum_j M_{ij} \left(dw_{ij}^\ell r_{ij} + w_{ij}^\ell dr_{ij}\right). \tag{A62}$$

This expression has two parts. The term $dw_{ij}^\ell r_{ij}$ captures how the learned scalar weight changes, and the term $w_{ij}^\ell dr_{ij}$ captures how the relative displacement itself changes. The node message aggregation is

$$\bar{m}_i^\ell = \frac{1}{N} \sum_j M_{ij} m_{ij}^\ell, \qquad d\bar{m}_i^\ell = \frac{1}{N} \sum_j M_{ij} dm_{ij}^\ell. \tag{A63}$$

The node network receives $(h_i^\ell, \bar{m}_i^\ell, \tau)$ and JVP input $(dh_i^\ell, d\bar{m}_i^\ell, 0)$. If the node MLP returns Δh_i^ℓ and $d\Delta h_i^\ell$, then

$$h_i^{\ell+1} = h_i^\ell + \Delta h_i^\ell, \qquad dh_i^{\ell+1} = dh_i^\ell + d\Delta h_i^\ell. \tag{A64}$$

The EGNN repeats this process layer by layer: compute pair geometry, run the edge network, update coordinates, aggregate messages, and update node features, while carrying the matching directional derivatives alongside the ordinary values. After L layers, the code returns

$$f_\theta(x, \tau) = \frac{1}{L} \sum_{\ell=1}^{L} \Delta x^\ell, \qquad J_{f_\theta}(x, \tau)v = \frac{1}{L} \sum_{\ell=1}^{L} d\Delta x^\ell. \tag{A65}$$

This recursion gives the full analytic JVP that the `hutchinson_jvp` trace mode uses.

7. Variance-Preserving Diffusion Score Target

DDPM defines a discrete noising Markov chain

$$q(x_k | x_{k-1}) = \mathcal{N}\left(x_k; \sqrt{1 - \beta_k}\, x_{k-1}, \beta_k I\right). \tag{A66}$$

Here β_k is the noise variance added at step k. Choosing small β_k makes each transition a small perturbation, so many steps produce a smooth path from data to noise. Equivalently,

$$x_k = \sqrt{1 - \beta_k}\, x_{k-1} + \sqrt{\beta_k}\, \epsilon_k, \qquad \epsilon_k \sim \mathcal{N}(0, I). \tag{A67}$$

To obtain a continuous Markov noising process, take a small timestep Δt and set $\beta_k = \beta(t)\Delta t$. The deterministic factor $\sqrt{1 - \beta_k}$ slightly shrinks the current state, while

the random term $\sqrt{\beta_k}\epsilon_k$ adds Gaussian noise with variance β_k. Using $\sqrt{1-\beta(t)\Delta t} = 1 - \frac{1}{2}\beta(t)\Delta t + o(\Delta t)$ gives

$$x_{t+\Delta t} - x_t = -\frac{1}{2}\beta(t)x_t\,\Delta t + \sqrt{\beta(t)\Delta t}\,\epsilon_t + o(\Delta t). \tag{A68}$$

The limit $\Delta t \to 0$ gives the VP forward SDE

$$dx_t = -\frac{1}{2}\beta(t)x_t\,dt + \sqrt{\beta(t)}\,dW_t. \tag{A69}$$

This limit uses the Brownian scaling $dW_t \approx \sqrt{\Delta t}\epsilon_t$. This continuous process keeps the DDPM Markov structure while giving an SDE whose marginal transition density we can compute in closed form. Define

$$B(t) = \int_0^t \beta(s)\,ds. \tag{A70}$$

Itô's product rule, which follows from Itô's lemma with $\psi(x,t) = e^{B(t)/2}x$, shows that multiplying by the integrating factor $e^{B(t)/2}$ removes the deterministic decay:

$$d\left[e^{B(t)/2}x_t\right] = e^{B(t)/2}\sqrt{\beta(t)}\,dw_t. \tag{A71}$$

Integrating from 0 to t gives

$$\begin{aligned} x_t = {} & e^{-B(t)/2}x_0 \\ & + \int_0^t e^{-(B(t)-B(s))/2}\sqrt{\beta(s)}\,dW_s. \end{aligned} \tag{A72}$$

Taking conditional expectation over the Brownian path gives

$$\mathbb{E}[x_t|x_0] = e^{-B(t)/2}x_0. \tag{A73}$$

The stochastic integral has mean zero because Itô increments have mean zero. Write that stochastic integral as

$$\eta_t = \int_0^t \phi(s)\,dW_s, \qquad \phi(s) = e^{-(B(t)-B(s))/2}\sqrt{\beta(s)}. \tag{A74}$$

Then $x_t = e^{-B(t)/2}x_0 + \eta_t$ and $\mathbb{E}[\eta_t|x_0] = 0$. The conditional variance follows by subtracting the squared conditional mean:

$$\begin{aligned} \mathsf{Var}\,[x_t|x_0] &= \mathbb{E}\left[x_t^2\middle|x_0\right] - \mathbb{E}\left[x_t|x_0\right]^2 \\ &= \mathbb{E}\left[\left(e^{-B(t)/2}x_0 + \eta_t\right)^2\middle|x_0\right] - e^{-B(t)}x_0^2 \\ &= e^{-B(t)}x_0^2 + 2e^{-B(t)/2}x_0\mathbb{E}[\eta_t|x_0] \\ &\quad + \mathbb{E}[\eta_t^2|x_0] - e^{-B(t)}x_0^2 \\ &= \mathbb{E}\left[\eta_t^2|x_0\right] \\ &= \int_0^t \phi^2(s)\,ds \\ &= e^{-B(t)}\int_0^t e^{B(s)}\beta(s)\,ds \\ &= e^{-B(t)}\int_0^t \frac{d}{ds}e^{B(s)}\,ds \\ &= 1 - e^{-B(t)}. \end{aligned} \tag{A75}$$

The conditional noise variance $1 - e^{-B(t)}$ complements the squared signal coefficient $e^{-B(t)}$, which motivates the name variance-preserving. For a fixed initial condition x_0, the marginal transition distribution takes Gaussian form

$$p_{0t}(x_t|x_0) = \mathcal{N}\left(x_t; \mu(t)x_0, \sigma^2(t)I\right). \tag{A76}$$

We can write the solution as

$$\begin{aligned} x_t &= \mu(t)x_0 + \sigma(t)\epsilon, \\ \epsilon &\sim \mathcal{N}(0, I), \end{aligned} \tag{A77}$$

where

$$\begin{aligned} \mu(t) &= e^{-B(t)/2}, \\ \sigma^2(t) &= 1 - \mu^2(t). \end{aligned} \tag{A78}$$

For the linear schedule $\beta(t) = \beta_{\mathsf{min}} + t(\beta_{\mathsf{max}} - \beta_{\mathsf{min}})$,

$$\begin{aligned} \int_0^t \beta(s)\,ds = {} & \beta_{\mathsf{min}}t \\ & + \frac{1}{2}\left(\beta_{\mathsf{max}} - \beta_{\mathsf{min}}\right)t^2, \end{aligned} \tag{A79}$$

which gives the $\mu(t)$ and $\sigma(t)$ expressions used in the main text. The conditional log density satisfies

$$\begin{aligned} \log p_{0t}(x_t|x_0) = {} & -\frac{1}{2\sigma^2(t)}\left\|x_t - \mu(t)x_0\right\|^2 \\ & + C(t), \end{aligned} \tag{A80}$$

where $C(t)$ does not depend on x_t. Differentiating with respect to x_t gives

$$\nabla_{x_t}\log p_{0t}(x_t|x_0) = -\frac{x_t - \mu(t)x_0}{\sigma^2(t)}. \tag{A81}$$

Using $x_t - \mu(t)x_0 = \sigma(t)\epsilon$ gives the denoising score target

$$\nabla_{x_t}\log p_{0t}(x_t|x_0) = -\frac{\epsilon}{\sigma(t)}. \tag{A82}$$

The quantity on the left is the conditional score,

$$s_{0t}(x_t|x_0) = \nabla_{x_t} \log p_{0t}(x_t|x_0), \tag{A83}$$

which is the score of the noised distribution when the clean starting point x_0 is known. The reverse SDE, however, requires the marginal score $\nabla_{x_t} \log p_t(x_t)$ after averaging over all possible clean configurations that could have produced the same noisy point. These two scores connect through

$$\begin{aligned} \nabla_{x_t} \log p_t(x_t) &= \frac{\nabla_{x_t} p_t(x_t)}{p_t(x_t)} \\ &= \frac{\int p_0(x_0) \nabla_{x_t} p_{0t}(x_t|x_0)\, dx_0}{p_t(x_t)} \\ &= \mathbb{E}\left[\nabla_{x_t} \log p_{0t}(x_t|x_0)|x_t\right]. \end{aligned} \tag{A84}$$

Thus training on pairs (x_0, x_t) with the conditional target $-\epsilon/\sigma(t)$ teaches the network the conditional expectation that equals the marginal score needed by the reverse sampler. The diffusion network $s_\theta(x_t, t)$ learns to approximate this score from noisy samples. The training objective weights the squared score-matching error by $\sigma^2(t)$:

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}\left[\sigma^2(t) \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma(t)} \right\|_2^2\right]. \tag{A85}$$

8. Fokker-Planck Time Reversal

The score target above gives the quantity that the reverse-time sampler needs. To see why the score appears in the reverse SDE, start from a general forward diffusion

$$dX_t = f(X_t, t)\, dt + g(t)\, dW_t \tag{A86}$$

with density $p_t(x)$. Its Fokker-Planck equation is

$$\frac{\partial p_t}{\partial t} = -\nabla_x \cdot [f(x,t)p_t(x)] + \frac{1}{2} g^2(t) \Delta_x p_t(x). \tag{A87}$$

Equivalently, the density evolves by a probability current,

$$\begin{aligned} \frac{\partial p_t}{\partial t} &= -\nabla_x \cdot J_t(x), \\ J_t(x) &= f(x,t)p_t(x) - \frac{1}{2} g^2(t) \nabla_x p_t(x). \end{aligned} \tag{A88}$$

The first term transports density through the deterministic drift. The second term is the diffusive current down the density gradient.

Now define the backward-time process $Y_s = X_{T-s}$ and write its density as $q_s(y) = p_{T-s}(y)$. Because $t = T - s$,

$$\frac{\partial q_s}{\partial s} = -\frac{\partial p_t}{\partial t} = \nabla_y \cdot J_t(y). \tag{A89}$$

If Y_s follows

$$dY_s = \tilde{f}(Y_s, s)\, ds + g(T-s)\, d\bar{W}_s, \tag{A90}$$

then its Fokker-Planck equation is

$$\begin{aligned} \frac{\partial q_s}{\partial s} &= -\nabla_y \cdot \left[\tilde{f}(y,s) q_s(y)\right] \\ &\quad + \frac{1}{2} g^2(t) \Delta_y q_s(y). \end{aligned} \tag{A91}$$

Substituting $q_s(y) = p_t(y)$ into this backward-time Fokker-Planck equation gives

$$\begin{aligned} \frac{\partial q_s}{\partial s} &= -\nabla_y \cdot \left[\tilde{f}(y,s) p_t(y)\right] \\ &\quad + \frac{1}{2} g^2(t) \Delta_y p_t(y). \end{aligned} \tag{A92}$$

The reversed density must also satisfy $\partial_s q_s = \nabla_y \cdot J_t(y)$. Using the forward current $J_t(y) = f(y,t)p_t(y) - \frac{1}{2} g^2(t) \nabla_y p_t(y)$ gives

$$\begin{aligned} \frac{\partial q_s}{\partial s} &= \nabla_y \cdot \left[f(y,t)p_t(y) - \frac{1}{2} g^2(t) \nabla_y p_t(y)\right] \\ &= \nabla_y \cdot [f(y,t)p_t(y)] - \frac{1}{2} g^2(t) \Delta_y p_t(y). \end{aligned} \tag{A93}$$

Equating the two expressions for $\partial_s q_s$ and moving the diffusion term gives

$$-\nabla_y \cdot \left[\tilde{f}(y,s) p_t(y)\right] = \nabla_y \cdot \left[f(y,t)p_t(y) - g^2(t) \nabla_y p_t(y)\right]. \tag{A94}$$

Thus the reverse current can be written as

$$\tilde{f}(y,s) p_t(y) = -f(y,t)p_t(y) + g^2(t) \nabla_y p_t(y). \tag{A95}$$

Dividing by $p_t(y)$ converts the density-gradient term into the score:

$$\tilde{f}(y,s) = -f(y,t) + g^2(t) \nabla_y \log p_t(y), \qquad t = T - s. \tag{A96}$$

Thus the reverse dynamics in the increasing backward-time variable s use $-f + g^2 \nabla \log p_t$. This equation describes $Y_s = X_{T-s}$, so increasing s means decreasing the original time t. Equivalently, $ds = -dt$. If we rewrite the same reverse process using the original variable X_t and a negative timestep $dt < 0$, the sign of the drift term changes:

$$\tilde{f}\, ds = \left[-f + g^2 \nabla \log p_t\right](-dt) = \left[f - g^2 \nabla \log p_t\right] dt. \tag{A97}$$

The Brownian motion also belongs to the reversed filtration, so we write it as $\bar{W}_t$ to distinguish reverse-time noise from the forward Brownian path W_t. The barred process has the same Brownian increment law, but a reverse simulation draws reverse-time increments rather than reusing the original forward increments. With this notation, the equivalent reverse-time SDE in the original time variable is

$$dX_t = \left[f(X_t, t) - g^2(t) \nabla_x \log p_t(X_t)\right] dt + g(t)\, d\bar{W}_t. \tag{A98}$$

The reverse VP sampler below substitutes the VP drift, diffusion coefficient, and learned score into this equation.

9. Reverse-Time VP Sampler

The Fokker-Planck time-reversal derivation above applies to the VP forward diffusion

$$dx_t = f(x_t, t)\,dt + g(t)\,dw_t, \tag{A99}$$

with density $p_t(x)$. Keeping the original time variable and stepping with $dt < 0$ gives the reverse drift

$$f(x_t, t) - g^2(t)\nabla_{x_t} \log p_t(x_t) \tag{A100}$$

inside the reverse-time SDE. For the VP process,

$$\begin{aligned} f(x_t, t) &= -\frac{1}{2}\beta(t)x_t, \\ g^2(t) &= \beta(t). \end{aligned} \tag{A101}$$

Substituting these coefficients into the reverse-time formula gives

$$dX_t = \left[-\frac{1}{2}\beta(t)X_t - \beta(t)\nabla_x \log p_t(X_t)\right] dt + \sqrt{\beta(t)}\,d\bar{W}_t, \tag{A102}$$

where numerical sampling uses $dt < 0$. Replacing the unknown score $\nabla_x \log p_t(x)$ with the learned score model $s_\theta(x, t)$ gives the model reverse drift

$$-\frac{1}{2}\beta(t)x_t - \beta(t)s_\theta(x_t, t). \tag{A103}$$

Because the implementation steps from a larger time t to a smaller time $t - \Delta t$, it applies this reverse SDE with a negative time increment. Equivalently, we write the Euler-Maruyama update as

$$\begin{aligned} x_{t-\Delta t} = x_t &- \left[f(x_t, t) - g^2(t)s_\theta(x_t, t)\right]\Delta t \\ &+ g(t)\sqrt{\Delta t}\,z, \qquad z \sim \mathcal{N}(0, I). \end{aligned} \tag{A104}$$

Substituting the VP coefficients gives

$$\begin{aligned} x_{t-\Delta t} = x_t &+ \left[\frac{1}{2}\beta(t)x_t + \beta(t)s_\theta(x_t, t)\right]\Delta t \\ &+ \sqrt{\beta(t)\Delta t}\,z. \end{aligned} \tag{A105}$$

The code recenters the state after each step so the sampler remains in the same centered coordinate subspace used during training. It also omits the final noise draw on the last step to reduce endpoint noise near $t = 10^{-4}$.